

CloudMart: Production-Grade Microservices Platform on Cloud-Managed Kubernetes

A Comprehensive Cloud Infrastructure Design, Deployment & Security Project

1. Assignment Overview

This group assignment challenges you to apply the full range of cloud infrastructure skills acquired throughout IS 4630 in a realistic, industry-grade scenario. You will design, containerise, deploy, secure, and optimise a multi-service e-commerce application using a **managed Kubernetes service** on the cloud provider of your group's choice — AWS, Google Cloud Platform (GCP), or Microsoft Azure.

Throughout this document, cloud services are described using **generic terms** (e.g. "managed Kubernetes," "container registry"). Refer to **Appendix A: Cloud Provider Service Mapping** for the exact service names on each provider.

The assignment mirrors the kind of infrastructure project you will encounter in industry: ambiguous requirements, real cost constraints, regulatory-style compliance expectations, and the need to present technical decisions clearly to a non-technical audience.

1.1 Scenario: CloudMart

CloudMart is a growing online retail startup that has outgrown its monolithic deployment on a single virtual machine. The founding engineering team has decomposed their application into five microservices and needs a team of cloud engineers to:

- Design and implement a production-ready virtual network with correct subnet, routing, and firewall/security group architecture
- Containerise each microservice and publish images to a managed container registry
- Deploy all services to a managed Kubernetes cluster with appropriate Kubernetes primitives
- Implement security controls at every layer — identity, network, container, and application
- Build an automated CI/CD pipeline that deploys on every code push
- Configure observability (metrics, logs, alerts) so the team can detect and respond to incidents
- Manage and optimise cloud costs using FinOps practices
- Prepare a disaster recovery strategy with defined RTO and RPO targets

2. The CloudMart Application

CloudMart's engineering team has provided the following five microservices. Starter code (fully working services with in-memory data stores) is available in the private GitHub repository:

Your group is responsible for extending the provided services with cloud-managed backends (replacing the in-memory stores), containerising each service, and deploying them to your managed Kubernetes cluster.

Service	Port	Language	Responsibilities
product-service	8001	Python / Flask	Product catalogue CRUD; search; category management; image URL references stored in a managed NoSQL database
order-service	8002	Node.js / Express	Create, read, and update orders; emit order events to a managed message queue; read product availability from product-service
user-service	8003	Python / Flask	User registration, login (JWT), profile management; passwords hashed with bcrypt; data in a managed relational database (PostgreSQL)
notification-service	8004	Node.js	Consumes order events from the message queue; sends email notifications via a cloud email/messaging service; no inbound HTTP traffic
frontend	80 / 443	React / Nginx	Single-page application served by Nginx; calls backend services via a cloud load balancer / Ingress; SSL termination at load balancer

2.1 Inter-Service Communication

Services communicate as follows — students must reflect these patterns in both their Kubernetes Service definitions and NetworkPolicy rules:

- **frontend** → product-service, order-service, user-service (HTTP via Ingress / cloud load balancer)
- **order-service** → product-service (internal ClusterIP), managed message queue (cloud-managed)
- **notification-service** → message queue (poll), email service (send) — no inbound HTTP
- **user-service** → managed relational database (data subnet, port 5432)
- **product-service** → managed NoSQL database (via private connectivity / endpoint)

2.2 Required Cloud Managed Services

Your deployment must use the equivalent managed services on your chosen provider (see **Appendix A** for the exact service names). Free Tier / minimal-cost usage is expected:

- **Managed Kubernetes** — Kubernetes control plane (1 cluster, 2–3 worker nodes)
- **Container Registry** — Container image registry (one repository per service)
- **Managed Relational Database** (PostgreSQL, smallest instance) — user-service data store
- **Managed NoSQL Database** — product-service catalogue (on-demand/serverless pricing)
- **Managed Message Queue** — order event queue (standard queue, free tier where available)
- **Cloud Email / Messaging Service** — email notifications (sandbox/test mode acceptable)
- **Cloud Load Balancer** (via Ingress controller) — Ingress for frontend
- **Cloud Monitoring Service** — metrics, logs, container insights
- **Cloud Threat Detection** — security monitoring (free trial where available)
- **Object Storage** — Terraform state backend + static assets
- **Key Management Service** — encryption key for database and secrets
- **Secret Management Service** — database credentials and API keys

3. Technical Requirements

Each requirement below maps to one or more lectures in IS 4630. Requirements are marked as **Mandatory (M)**, **Recommended (R)**, or **Distinction (D)** to help you prioritise effort.

3.1 Networking & Virtual Network [Lectures 3, 4]

- **[M]** Virtual network with a large CIDR block (e.g. /16), spanning at least 2 availability zones (or equivalent regions/zones)
- **[M]** Three-tier subnet design: public, private application, private data — one subnet per tier per zone
- **[M]** Internet gateway for public subnets; NAT gateway (or equivalent) in public subnet(s) for private subnet egress
- **[M]** Route tables correctly configured for each subnet tier
- **[M]** Firewall rules / security groups for: load balancer, Kubernetes worker nodes, database, and a bastion host
- **[M]** Firewall rules documented with justification (principle of least privilege)
- **[R]** Private connectivity (VPC endpoint / Private Service Connect / Private Endpoint) for NoSQL database — eliminates NAT data-transfer cost
- **[R]** Private connectivity for secret management service — keeps secrets traffic off the public internet
- **[D]** Network flow logs enabled, sent to cloud logging, with at least one analytics query analysing rejected traffic

3.2 Containerisation [Lecture 5]

- **[M]** Dockerfile for each of the five services, following best practices:
 - Multi-stage build to minimise final image size (build stage vs runtime stage)
 - Non-root USER specified in every Dockerfile
 - Minimal base image (python:3.11-slim, node:20-alpine, nginx:alpine)
 - HEALTHCHECK instruction defined
- **[M]** Images pushed to your cloud provider's container registry; lifecycle/cleanup policy retaining only the last 10 images per repository
- **[M]** .dockerignore file present for each service, excluding node_modules, pycache, .git, and test directories
- **[R]** Docker Compose file for local development (mirrors production service communication)
- **[D]** Container image vulnerability scan using Trivy (or provider-native scanner) in CI; pipeline fails on CRITICAL findings

3.3 Kubernetes on Managed K8s [Lectures 6, 7]

- **[M]** All five services deployed as Kubernetes Deployments with:
 - Minimum 2 replicas for production services (product, order, user, frontend)
 - Resource requests and limits defined for every container
 - liveness and readiness probes configured
 - Rolling update strategy with maxSurge: 1, maxUnavailable: 0
- **[M]** Kubernetes Services: ClusterIP for backend services; Ingress for frontend via cloud load balancer (using provider's Ingress controller)
- **[M]** Ingress resource configured with appropriate annotations for your provider's load balancer controller
- **[M]** Horizontal Pod Autoscaler (HPA) for product-service and order-service targeting 60% CPU utilisation
- **[M]** Namespaces: **cloudmart-prod** (production), **cloudmart-staging** (staging)
- **[M]** ConfigMaps for non-sensitive configuration (database hostnames, queue URLs, feature flags)
- **[M]** Kubernetes Secrets for sensitive values (injected from secret management service via Secrets Store CSI Driver or External Secrets Operator)
- **[R]** Pod Disruption Budget (PDB) for each production deployment ensuring minimum 1 replica available
- **[R]** Cluster autoscaler (or equivalent) deployed and demonstrated scaling out under load
- **[D]** Event-driven autoscaler (e.g. KEDA) scaling notification-service based on message queue depth

3.4 Security [Lectures 4, 7, 8]

- **[M]** Default-deny NetworkPolicy applied to the **cloudmart-prod** namespace
- **[M]** Explicit allow NetworkPolicy rules for each permitted communication path.

- **[M]** Workload identity (cloud IAM ↔ Kubernetes service account binding) with separate, minimal IAM role/service account per service:
 - product-service: read/write access to NoSQL database (products table/collection only)
 - order-service: send/receive access to message queue (orders queue only)
 - notification-service: receive/delete from queue, send email — only
 - user-service: read access to database credentials in secret management service only
- **[M]** Kubernetes node IAM role/service account must NOT have excessive permissions (no admin/owner access)
- **[M]** Database encrypted at rest (managed key); data in transit via SSL enforced
- **[M]** Instance metadata service hardened (IMDSv2 on AWS, or equivalent protection on GCP/Azure)
- **[R]** Cloud threat detection enabled; present at least one finding (simulated or real) and the response action
- **[R]** Web application firewall (WAF) attached to the load balancer with managed rule sets enabled
- **[D]** Policy engine (OPA/Gatekeeper or Kyverno) preventing containers running as root, preventing privileged containers, and enforcing image pull from your container registry only

3.5 CI/CD Pipeline [Lecture 9]

- **[M]** Git repository with branch strategy: **main** (production), **develop** (staging), **feature/*** branches
- **[M]** CI pipeline triggered on every push to main and develop, performing:
 - Lint and unit tests for each service
 - Docker image build and push to container registry (tagged with commit SHA)
 - Trivy vulnerability scan (CRITICAL findings = pipeline failure)
 - Kubernetes manifest validation (kubeval or kubeconform)
- **[M]** CD pipeline deploying to **cloudmart-staging** on develop merge, **cloudmart-prod** on main merge
- **[M]** Deployment strategy: rolling update with health-check gate before promoting to 100% of replicas
- **[R]** Manual approval gate in the pipeline before deploying to production
- **[R]** Post-deployment smoke test (HTTP health check against each service readiness endpoint)
- **[D]** Canary deployment for product-service using Argo Rollouts or Flagger; monitoring metric gate halts rollout if error rate exceeds 1%

3.6 Observability [Lectures 9, 11]

- **[M]** Container/Kubernetes monitoring enabled on the cluster (provider's container insights or Prometheus + Grafana)

- **[M]** Application logs from all five services shipped to cloud logging (one log group/stream per service)
- **[M]** Monitoring dashboard showing: CPU and memory per service, request rate, error rate, queue depth, database connections
- **[M]** Alert/alarm on product-service error rate > 5% over 5 minutes, triggering notification (email/SMS/webhook)
- **[R]** Custom application metric: order throughput (orders per minute) published to monitoring from order-service
- **[D]** Distributed tracing enabled (cloud-native tracing or Jaeger/Zipkin); trace a single order from frontend through order-service to queue

3.7 Infrastructure as Code [Lectures 9, 11]

- **[M]** All cloud infrastructure (virtual network, subnets, firewall rules, Kubernetes cluster, database, NoSQL database, queue, container registry) provisioned using Terraform
- **[M]** Terraform state stored remotely with state locking (object storage + lock table/mechanism)
- **[M]** Variables parameterised for environment (staging vs production)
- **[R]** Kubernetes manifests managed with Helm charts (one chart per service with values files for staging and production)
- **[D]** GitOps using ArgoCD: ArgoCD watches the Git repository and automatically syncs Kubernetes state

3.8 Cost Management [Lecture 10]

- **[M]** All cloud resources tagged/labelled with: `Project=cloudmart, Environment=(prod|staging), Team=(your-group-id), Owner=(email)`
- **[M]** Cost report from provider's cost management tool showing daily spend by service/tag
- **[M]** Monthly budget alert configured: notify via email when spend exceeds your threshold
- **[M]** Architecture Decision Record (ADR) documenting the choice of Kubernetes node instance/machine type, including cost vs performance analysis
- **[R]** Provider's compute sizing recommendations reviewed; justify whether each recommendation was accepted or rejected
- **[R]** Calculate the unit economics metric: estimated infrastructure cost per 1,000 orders processed
- **[D]** Committed use / savings analysis: model the 1-year saving if production nodes were covered by a committed-use discount vs on-demand pricing

3.9 Disaster Recovery [Lecture 11]

- **[M]** Define RTO and RPO targets for the CloudMart platform with business justification
- **[M]** Database automated backups enabled with 7-day retention; demonstrate point-in-time recovery procedure

- **[M]** Kubernetes manifest backup: all Kubernetes resources exported to Git (Velero or `kubect1 get all -o yaml` piped to Git)
- **[R]** Multi-zone database deployment with demonstrated automatic failover
- **[D]** DNS health-check configured; demonstrate DNS failover to a static error page hosted in object storage when the load balancer health check fails

4. Architecture Diagram Requirements

Your report must include the following architecture diagrams. Tools such as draw.io (diagrams.net), Lucidchart, or your provider's architecture icon set are recommended.

Diagram	Required Contents
D1: Network Architecture	Virtual network CIDR, all subnets with CIDRs, internet gateway, NAT gateway(s), DNS, load balancer, private endpoints, K8s node placement, database placement
D2: Kubernetes Architecture	K8s control plane, worker nodes, all 5 Deployments with replica count, Services, Ingress, HPA, namespaces boundary, cluster autoscaler
D3: Security Model	NetworkPolicy flows (which pods can talk to which), IAM/workload identity role-to-service mapping, encryption key usage, secret management, WAF position, threat detection scope
D4: CI/CD Pipeline	All pipeline stages from code push to production deployment, including test gates, image build, scan, approval gates, and rollback trigger
D5: Disaster Recovery	Backup strategy, database multi-zone failover path, Velero restore procedure, DNS failover configuration, estimated recovery timeline

5. Deliverables

5.1 GitHub Repository

Submit the URL of a public (or lecturer-accessible private) GitHub repository containing:

- Terraform code in `/infra/` directory
- Kubernetes manifests or Helm charts in `/k8s/` directory
- Application source code and Dockerfiles in `/services/<service-name>/` directories
- CI/CD pipeline configuration (`.github/workflows/` or equivalent)
- Architecture Decision Records in `/docs/adr/` (minimum 3 ADRs — see Section 6)
- README.md with: chosen cloud provider, deployment instructions, architecture summary, team members and individual contributions

5.2 Written Report (10–15 Pages)

Submit one PDF report per group. The report must follow the structure below. Page count excludes diagrams and appendices.

Report Section	Content Summary	Pages
1. Executive Summary	Project scope, chosen cloud provider and rationale, key architectural decisions, outcome metrics (cost, availability, security posture)	1
2. System Architecture	Architecture diagrams D1 and D2 with explanatory narrative; justification of key technology choices	2–3
3. Security Design	Threat model, network policy justification, workload identity design, diagram D3, any threat detection findings and response	2
4. DevOps & CI/CD	Pipeline design (diagram D4), deployment strategy rationale, infrastructure-as-code approach	1–2
5. Cost Analysis & FinOps	Actual cloud spend report (cost dashboard screenshot), unit economics, savings analysis, cost optimisation actions taken	1–2
6. Disaster Recovery Plan	RTO/RPO targets, backup strategy, diagram D5, test evidence (screenshots of backup/restore)	1
7. Architecture Decision Records	Three ADRs in Nygard format (see Section 6). At least one must cover a cost-architecture trade-off	1–2
8. Reflection & Lessons Learned	What worked well, what the team would do differently, one real industry case study that relates to a decision you made	1

5.3 Live Demo (20 Minutes)

Each group will present a live demonstration of their deployed CloudMart platform. The demo must show a working end-to-end user journey and evidence of each major technical area.

5.4 Individual Viva (10–15 Minutes per Student)

Following the group demo, each student will be questioned individually on the technical content of the project. Questions will be drawn from the technical requirements (Section 3) and the case studies in your report. The individual viva carries **40% of the total mark**, so every team member must understand the complete system — not just their own component. Students who cannot explain their group's technical decisions will receive a significantly reduced individual mark.

6. Architecture Decision Records (ADRs)

Your group must produce three ADRs using the Michael Nygard format. Each ADR must be:

- In Markdown format in your repository at `/docs/adr/ADR-NNN-title.md`
- Included (summarised) in Section 7 of your report
- Status: Accepted, Superseded, or Deprecated (with reason if superseded)

6.1 Required ADR Topics

You must write ADRs for the following decisions. Additional ADRs are encouraged but not required:

#	Decision Topic	Guidance
1	Kubernetes node instance/machine type selection	Compare at least 3 machine types from your provider (e.g., general-purpose vs compute-optimised vs ARM-based). Analyse cost, vCPU/RAM ratio, ARM savings. Justify final choice.
2	Database technology for user-service	Compare managed PostgreSQL vs managed NoSQL vs serverless relational for user data. Consider cost, consistency requirements, query patterns, and ops overhead.
3	Deployment strategy for product-service	Compare rolling update, blue/green, and canary. Justify chosen strategy given CloudMart's scale, RTO target, and team's operational maturity.

6.2 ADR Template

None

ADR-NNN: [Short descriptive title]

Status

Accepted | Superseded | Deprecated

Context

Describe the issue motivating this decision. Include relevant constraints, business requirements, and the problem you are trying to solve.

Decision

Describe the decision made. Be specific and unambiguous.

Consequences

What becomes easier or harder as a result of this decision?

List both positive and negative consequences.

Alternatives Considered

List at least 2 alternatives and why they were rejected.

7. Demo Script & Assessment Checkpoints

The 20-minute demo must cover the following checkpoints in order. Assessors will mark each checkpoint as Pass, Partial, or Fail based on what is demonstrated live.

#	Area	What to Demonstrate	Evidence
1	Infrastructure	kubectl get nodes — show 2+ nodes running; kubectl get pods -n cloudmart-prod — all pods Running/Ready	Terminal output
2	End-to-End Flow	Open CloudMart in browser: register a user, browse products, place an order, verify order confirmation email received	Live browser + email
3	Networking	Show virtual network / subnet layout in cloud console. Demonstrate that database is NOT accessible from the public internet (telnet/nc from external host fails)	Console + terminal
4	Security	kubectl get networkpolicy -n cloudmart-prod; show workload identity binding for one service in console; show threat detection dashboard	Terminal + console
5	CI/CD	Make a small code change (e.g., change homepage text), push to develop branch, show pipeline running in GitHub Actions (or equivalent), verify change appears in staging	GitHub + live site
6	Autoscaling	Run a simple load test (hey or k6) against product-service. Show HPA scaling replicas in kubectl get hpa -w. Show monitoring metrics spike	Terminal + console
7	Observability	Open monitoring dashboard — show live metrics. Trigger a 404 error; show it appearing in log analytics query results	Console
8	Cost Management	Show cost dashboard with tags/labels applied; show the daily spend breakdown; show active budget alert. Present your unit economics calculation (cost per 1,000 orders)	Console + slide
9	Disaster Recovery	Show database automated backup is enabled; demonstrate restoring to a point-in-time in a test instance (not production). Show kubectl commands to restore from Velero backup	Console + terminal

10	Q&A	Assessor questions to the group on architectural decisions. Each student should be able to explain the components they were responsible for	Viva responses
----	-----	---	----------------

Appendix A: Cloud Provider Service Mapping

Use this table to identify the correct service name on your chosen provider. The assignment uses generic terms throughout — this appendix maps each to the specific service on AWS, GCP, and Azure.

Generic Term (used in assignment)	AWS	GCP	Azure
Managed Kubernetes	Elastic Kubernetes Service (EKS)	Google Kubernetes Engine (GKE)	Azure Kubernetes Service (AKS)
Container Registry	Elastic Container Registry (ECR)	Artifact Registry (or GCR)	Azure Container Registry (ACR)
Managed Relational Database	Amazon RDS (PostgreSQL)	Cloud SQL (PostgreSQL)	Azure Database for PostgreSQL Flexible Server
Managed NoSQL Database	Amazon DynamoDB	Firestore / Cloud Bigtable	Azure Cosmos DB (Table API or NoSQL)
Managed Message Queue	Amazon SQS	Cloud Pub/Sub	Azure Service Bus / Storage Queues
Cloud Email Service	Amazon SES	SendGrid (via Marketplace) or Gmail API	Azure Communication Services / SendGrid
Cloud Load Balancer	Application Load Balancer (ALB)	Cloud Load Balancing (HTTP(S))	Azure Application Gateway / Load Balancer
Ingress Controller	AWS Load Balancer Controller	GKE Ingress Controller / NGINX	AGIC (App Gateway Ingress) / NGINX
Cloud Monitoring	Amazon CloudWatch	Cloud Monitoring (Stackdriver)	Azure Monitor
Container Insights	CloudWatch Container Insights	GKE Monitoring (built-in)	Container Insights (Azure Monitor)
Cloud Logging	CloudWatch Logs	Cloud Logging (Stackdriver)	Azure Monitor Logs (Log Analytics)
Cloud Threat Detection	Amazon GuardDuty	Security Command Center (SCC)	Microsoft Defender for Cloud
Web Application Firewall	AWS WAF	Cloud Armor	Azure WAF (on Application Gateway)
Object Storage	Amazon S3	Cloud Storage (GCS)	Azure Blob Storage

Key Management Service	AWS KMS	Cloud KMS	Azure Key Vault
Secret Management	AWS Secrets Manager	Secret Manager	Azure Key Vault (Secrets)
Identity & Access Management	AWS IAM	Google Cloud IAM	Azure AD / Entra ID + RBAC
Workload Identity	IAM Roles for Service Accounts (IRSA)	Workload Identity Federation	Azure Workload Identity (AAD Pod Identity v2)
DNS Service	Route 53	Cloud DNS	Azure DNS
NAT Gateway	NAT Gateway	Cloud NAT	NAT Gateway
Virtual Network	VPC (Virtual Private Cloud)	VPC (Virtual Private Cloud)	VNet (Virtual Network)
Firewall Rules	Security Groups + NACLs	Firewall Rules (VPC)	Network Security Groups (NSG)
Private Connectivity	VPC Endpoints (Gateway/Interface)	Private Service Connect / Private Google Access	Private Endpoints / Service Endpoints
Network Flow Logs	VPC Flow Logs	VPC Flow Logs	NSG Flow Logs
Audit Logging	AWS CloudTrail	Cloud Audit Logs	Azure Activity Log
Cost Management	AWS Cost Explorer + Budgets	Cloud Billing + Budget Alerts	Azure Cost Management + Budgets
Compute Sizing Advisor	AWS Compute Optimizer	GCP Recommender	Azure Advisor
Committed Use Discounts	Reserved Instances / Savings Plans	Committed Use Discounts (CUDs)	Azure Reservations
Distributed Tracing	AWS X-Ray	Cloud Trace	Azure Application Insights
DDoS Protection	AWS Shield	Cloud Armor (DDoS)	Azure DDoS Protection